

Message_optimization

Введение

Общение в mesh-сетях практически всегда сопряжено с передачей данных. Публичные каналы, личные сообщения, новости, запросы, адверты, телеметрия и прочий служебный трафик постоянно создают нагрузку на радиоканал. Отдельного упоминания заслуживают и «радиохулиганы», но об этом как-нибудь в другой раз.

По мере роста количества устройств и пользователей, особенно, в крупных городах это может становиться заметной проблемой: ретрансляторы оказываются загружены, сообщения занимают эфирное время, а один и тот же пакет может многократно передаваться через сеть, увеличивая суммарную нагрузку.

Одним из способов управления такой нагрузкой является использование [регионов](#), позволяющих ограничить распространение трафика и не заставлять сообщения путешествовать по всей сети. Однако, оптимизация маршрутизации - далеко не единственный способ уменьшить нагрузку. Если посмотреть на проблему с другой стороны, возникает простой вопрос:

А что, если передавать то же самое сообщение, используя меньше данных?

Размер сообщения напрямую влияет на время его передачи и, следовательно, на занятость радиоканала. Чем меньше передаваемых данных, тем меньше времени занимает передача и тем больше сообщений потенциально может пройти через тот же канал. Под экологичным общением здесь понимается не выстраивание личных границ, а подход, при котором мы стараемся передавать необходимый смысл, используя минимально возможное количество данных и не создавая лишнюю нагрузку на mesh-сеть.

Цель данной работы - рассмотреть несколько способов уменьшения веса текстовых сообщений в mesh-сетях, оценить их эффективность и связанные с ними ограничения.

1. Матчасть

Как известно, размер символа зависит от используемой кодировки. Например, символы латинского алфавита в формате UTF-8, входящие в таблицу ASCII, кодируются одним байтом, тогда как символы кириллицы, как правило, занимают два байта.

Для количественного описания изменения размера сообщения введём **коэффициент сжатия**:

$$k = 1 - \frac{\text{Вес в новом формате}}{\text{Исходный вес}} = \frac{\sum_{i=1}^{33} b'_i}{\sum_{i=1}^{33} b_i}$$

где: (b_i) и (b'_i) - вес (i) -го символа в исходном и новом формате соответственно.

Однако данный коэффициент не учитывает частоту появления отдельных символов в языке. Поэтому для более устойчивой оценки введём **эффективный коэффициент сжатия**, учитывающий частотность появления каждого отдельного символа:

$$k_{eff} = 1 - \frac{\text{Вес в новом формате с учетом частотности буквы}}{\text{Исходный вес с учетом частотности буквы}} = 1 - \frac{\sum_{i=1}^{33} b'_i \cdot f_i}{\sum_{i=1}^{33} b_i \cdot f_i}$$

где:

- (b_i) - вес (i) -го символа в исходном формате;
- (b'_i) - вес (i) -го символа в новом формате;
- (f_i) - частотность появления (i) -го символа.

Таким образом, (k_{eff}) позволяет оценивать ожидаемое изменение объёма данных для текста, характерного для рассматриваемого языка. Во всех расчетах будем опираться на значения частотности, представленные в Табл. 1. *

Таблица 1.: [Частотность букв русского алфавита](#)

№	Буква	Частотность	Частотность %
1	о	0.1098	10.98%
2	е	0.0848	8.48%
3	а	0.0800	8.00%
4	и	0.0737	7.37%
5	н	0.0670	6.70%
6	т	0.0632	6.32%
7	с	0.0547	5.47%
8	р	0.0475	4.75%
9	в	0.0453	4.53%
10	л	0.0434	4.34%
11	к	0.0349	3.49%
12	м	0.0320	3.20%
13	д	0.0298	2.98%
14	п	0.0280	2.80%
15	у	0.0262	2.62%
16	я	0.0200	2.00%
17	ы	0.0190	1.90%
18	ь	0.0174	1.74%
19	г	0.0169	1.69%
20	з	0.0164	1.64%
21	б	0.0159	1.59%
22	ч	0.0145	1.45%
23	й	0.0121	1.21%
24	х	0.0097	0.97%

№	Буква	Частотность	Частотность %
25	ж	0.0094	0.94%
26	ш	0.0072	0.72%
27	ю	0.0064	0.64%
28	ц	0.0049	0.49%
29	щ	0.0036	0.36%
30	э	0.0033	0.33%
31	ф	0.0027	0.27%
32	ъ	0.0004	0.04%
33	ё	0.0001	0.01%

Для упрощения расчёта эффективного коэффициента был написан скрипт в Jupyter Notebook `k_eff.ipynb`. В нём значение **1** обозначает, что символ заменён на однобайтовый аналог, а **2** и более - что для символа нет подходящего аналога, следовательно кодируется двумя (или более) байтами. Кроме того, отдельно была вынесена функция для обработки дублирующихся символов, например, **ё** → **е** и **й** → **и**. В целом, данный код применим и для других алфавитных языков.

** По-хорошему, нужно в течение длительного времени мониторить общий чат выбранного протокола связи и уже затем делать выводы о частотностях букв. В рамках данного исследования это опущено и мы свято верим, что данные источника удовлетворительно экстраполируются на живое общение.*

2. Методы сжатия

2.1. Частичная транслитерация

2.1.1. Идея первая

Прежде всего, стоит разобраться с буквами, однозначно не имеющими прямых аналогов в латинском алфавите. К таким относятся:

в, д, ж, з, л, н, т, ф, ц, ч, ш, щ, ъ, ы, ь, э, ю, я.

Также сразу можно принять равными пары **«и»/«й»** и **«е»/«ё»**, записывая их как **«и»** и **«е»** соответственно. В современном русском письме точки над «ё» во многих случаях опускаются, а различие между «и» и «й» также часто не является критичным для понимания текста.

Далее рассмотрим оставшиеся буквы русского алфавита. Сразу, чисто визуально, напрашиваются следующие пары:

- а → a
- б → b
- г → r
- е → e

- и → u
- к → k
- м → m
- о → o
- п → n
- р → p
- с → c
- у → y
- х → x

Таким образом, уже на этом этапе покрывается **15 из 33** рассматриваемых букв.

Для более количественной оценки, здесь и далее, используем эффективный коэффициент сжатия:

$$k_{eff} = 1 - \frac{\sum_{i=1}^{33} b'_i \cdot f_i}{\sum_{i=1}^{33} b_i \cdot f_i}$$

С учётом частотности букв для рассмотренного набора соответствий получаем:

$$k_{eff} = 0.31$$

Или около **31%** экономии веса сообщения.

Съешь еще этих тягких французских булок, да выпей же чаю

На мой взгляд - более чем читаемо.

2.1.2. Идея вторая

На этом этапе я уже планировал переходить ко второй части, посвящённой полной транслитерации, однако мне не давали покоя буквы, описанные в самом начале предыдущего параграфа. И тут пришла следующая идея: ведь можно использовать не только строчные, но и заглавные буквы английского алфавита

Тогда появляются следующие соответствия:

- в → V
- н → H
- т → T

В результате получаем:

Съешь еще эTух тягких фpaнцyзских булок, да Bыпей же чаю

Выглядит не очень привычно. Зато такой подход позволяет увеличить степень сжатия - примерно до **40%**.

Затем я вспомнил, что цифры в ASCII также кодируются одним байтом. Это позволило найти ещё несколько соответствий:

- $з \rightarrow 3$
- $в \rightarrow 8$
- $ч \rightarrow 4$
- $б \rightarrow 6$

Наиболее естественным здесь выглядит использование **3** вместо **з**: визуальное сходство достаточно очевидно. В свою очередь, **8** вместо **в** и **4** вместо **ч** выглядят и читаются уже значительно хуже. Отдельно стоит отметить замену **б** на **6**: в большинстве шрифтов она выглядит вполне естественно и даже более узнаваемо, чем замена на **b**.

Съешь еще эТух мягких франЦузских булок, да Выпей же 4аю

Тем не менее, если рассматривать их исключительно как способ уменьшения объёма передаваемых данных, такие замены, по расчётам, позволяют достичь рекордных **42%** экономии относительно исходного представления.

На этом этапе, конечно, можно продолжать поиск новых соответствий. Например, букву **ш** можно заменить на **w**, но уже на данном этапе читаемость заметно проседает

2.1.3. Вывод

На мой взгляд лучшим решением в этом блоке будет:

1. Замены из [2.1.1. Идея первая](#).
2. Некоторые замены из [2.1.2. Идея первая](#), а именно: $б \rightarrow 6$, $з \rightarrow 3$, $н \rightarrow H$, $т \rightarrow T$

В результате получаем:

- $а \rightarrow a$
- $б \rightarrow 6$
- $в \rightarrow B$
- $г \rightarrow r$
- $е \rightarrow e$
- $з \rightarrow 3$
- $и \rightarrow u$
- $к \rightarrow k$
- $м \rightarrow m$
- $н \rightarrow H$
- $о \rightarrow o$
- $п \rightarrow n$
- $р \rightarrow p$
- $с \rightarrow c$
- $т \rightarrow T$
- $у \rightarrow y$
- $х \rightarrow x$

Или если представить это в излюбленной фразе всех дизайнеров:

Съешь еще эТих мягких фраНцузских булок, да Выпей же чаю

Что эквивалентно сжатию примерно на 40%.

2.2. Полная транслитерация

2.2.1. Идея первая

Согласно анализу [литературных данных](#), одним из наиболее экономичных вариантов транслитерации русского текста является транслитерация, используемая при оформлении заграничного паспорта. Для записи русского текста используются только символы, представимые в однобайтовой кодировке ASCII и, соответственно, занимающие один байт в UTF-8.

Таблица 2.: [Русские буквы и их транслитерация в формате заграничного паспорта](#)

Буква	Транслит
а	a
б	b
в	v
г	g
д	d
е	e
ё	e
ж	zh
з	z
и	i
й	i
к	k
л	l
м	m
н	n
о	o
п	p
р	r
с	s
т	t
у	u
ф	f
х	kh
ц	ts
ч	ch
ш	sh

Буква	Транслит
щ	shch
ъ	ie/нет
ы	у
ь	нет
э	е
ю	yu
я	ia

Siेशesh eshche etikh miagkikh frantsuzskikh bulok, da vypei chaiu

При использовании такого метода "сжатия" удастся достичь уменьшения веса на **48%**.

2.2.2. Идея вторая

В глаза сразу бросается буква **щ**, которая при транслитерации занимает сразу 4 байта, в то время как её ближайший аналог **ш** - всего 2 байта. Однако, согласно расчётам, эта замена не вносит существенного вклада в общее сжатие из-за низкой частотности буквы.

Куда более интересным может являться решение заменить все 2 байтные буквы на их 2 байтный аналог, но уже в русском языке:

Съesh еще etix miagkiъ franцyзskix bulok, da vypei je чаю

2.2.3. Вывод

Результат, на мой взгляд спорный. Транслитерация и без того заметно ухудшает читаемость текста (если ты не пограничник). В данном случае ситуация дополнительно осложняется тем, что после преобразования в тексте сохраняются отдельные кириллические символы. Сжатие в этом случае, напомним, остается на уровне **47%**.

2.3. За пределами транслитерации (шифры и коды)

Если выйти за лингвистические рамки, то помимо транслитерации и других языковых преобразований можно придумать ещё множество способов оптимизации сообщений.

Например, можно вообще отказаться от непосредственной передачи текста и заранее договориться (что, чаще всего, самое сложное) о **словаре команд**. Каждому коду назначается определённое значение: от простого «Привет!» до целого типового предложения.

При использовании одного байта можно закодировать **256 различных значений**: от **0x00** до **0xFF**. В результате такой словарь позволит передавать всего одним байтом до 256 типовых сообщений.

Причем, такие сообщения вполне могут быть в виде тега, снабженного дополнительной информацией, например:

Код	Значение
00 <Lat> <Lon>	SOS <координаты>
01	Привет!
02	Как дела?
03	Всё хорошо
04	Где ты?
05 <Lat> <Lon>	Нахожусь на позиции <координаты>
06 <Lat> <Lon>	Направляюсь на позицию <координаты>
07	Я на месте
08 <t>	Буду через <t> минут
09	Не могу сейчас ответить
10 <T> <P> <H>	Погодные данные: <T> °C, <P> гПа, <H> %
11 <HH:MM>	Принял сообщение от <HH:MM>
67	Анекдот про листок бумаги
99	Не хочу с вами работать
...	...

3. Вывод

3.1. Частичный транслит

В результате небольших вычислений и чуть больших раздумий вариант транслита, описанный в [2.1.3. Вывод](#) (или приближенные к нему варианты) кажутся наиболее оптимальными.

Преимущества:

1. Незначительно ограничивает читаемость сообщений.
2. Позволяет сократить вес сообщения (очень приблизительно) на 40%.
3. Не требует специального словаря или предварительного согласования - правила преобразования понятны непосредственно из текста.
4. Достаточно широкое распространение. В Москве и МО (сеть MeshCore) есть около 10 энтузиастов, которые на постоянной основе используют схожий способ сжатия.

Недостатки:

1. Скорость чтения такого рода сообщений в любом случае снижается
2. Такого рода преобразования могут оказаться слабочитаемыми или не читаемыми вовсе при использовании кастомных шрифтов или клиентов mesh-сетей.
3. Сокращение в 40% - ориентировочное значение. Оно может варьироваться в зависимости от того какие символы наиболее часто применяются в каналах и личных сообщениях.

3.2. Полный транслит

В свою очередь полный транслит обладает рядом других преимуществ

1. Больше сжатие (48% против 40%)
2. Единообразие алфавита. В случае с частичным транслитом правила перевода достаточно размыты, для полного транслита существует единый формат
3. Меньшая вероятность проблем с клиентами и устройствами, которые некорректно отображают Unicode.

К недостаткам я бы отнес:

1. Заметно сниженная читаемость.
2. Заметно сниженная скорость чтения.
3. Необходимость в фиксированном формате.

3.3. Шифры и коды

Ввиду своей специфики шифры и другие методы кодирования принципиально отличаются от рассмотренных выше способов оптимизации текста. Их основная задача заключается не в сохранении читаемости, а в максимально компактном представлении заранее определённых сообщений.

Преимущества:

1. Максимальное сжатие - короткое значение может представлять целое предложение или набор параметров.
2. Предсказуемый размер сообщения - при использовании фиксированного формата легко заранее оценить объём передаваемых данных.
3. Отсутствие зависимости от языка - код `0x01` может означать «Привет!» независимо от того, на каком языке пользователь общается.
4. Удобство для типовых сообщений. Особенно эффективно для часто повторяющихся команд, статусов, телеметрии и координат.
5. Упрощенная автоматическая обработка.
6. Хорошая масштабируемость. Свободные значения можно постепенно заполнять новыми командами и типами сообщений.
7. Улучшенная защита от компроментации данных. Кодировка, согласованная между рядом лиц может повысить уровень защиты личных данных помимо кодировок используемого протокола связи
8. Снижение информативности перехваченного трафика. Согласованная между ограниченным кругом пользователей система кодов может затруднить понимание содержания сообщений посторонним наблюдателем и повысить уровень защиты данных.

Недостатки:

1. Необходимость в установлении единообразного формата в рамках страны или даже нескольких стран. И как следствие необходимость постоянной поддержки формата - при изменении значений новые сообщения могут неверно интерпретироваться

2. Ограниченное количество заранее определённых сообщений. Код практически бесполезен для повседневного общения.
3. Необходимость автоматической декодировки или ручного поиска по таблицам

Постскриптум

Совсем забыл про пробелы, ники и спецсимволы, видимо об этом когда-нибудь в другой раз :3. Но точно знаю, что если удалить пробел после знака препинания можно сэкономить пару байтов.

Пост²скриптум

Впервые пишу какую-то осмысленную статью и благодарю всех за прочтение, буду очень рад осмысленной критике и вашим предложениям!